

Table of contents

Appendix B. What should just work, and how to verify it	1
The one-time cloud setting everything depends on	1
The service map	1
Verify it, layer by layer	2
What works away from home (and what deliberately doesn't)	3
Reboot behavior	3

Appendix B. What should just work, and how to verify it

This is the at-a-glance reference for a finished build: every URL, what it's for, and the one-line check that proves it's healthy. Use it as a smoke test after a reboot, after an update, or any time something feels off, work top to bottom and the first failing row tells you which layer broke.

Throughout, substitute your own values where this guide uses placeholders: the Pi's Tailscale IP for `100.x.y.z` (run `tailscale ip -4` on the Pi) and your tailnet suffix for `your-tailnet.ts.net`.

The one-time cloud setting everything depends on

Almost everything below is configured *on the Pi* and works the moment the containers are up. The **single exception** is a setting in the Tailscale admin console that can't be done from the Pi, and until it's on, the `.home` names and tailnet-wide ad-blocking won't work on your *other* devices:

! Make Pi-hole your tailnet's DNS (admin console, one time)

In the Tailscale admin console (<https://login.tailscale.com>) → **DNS** page:

1. **Add nameserver** → **Custom**, enter the Pi's **Tailscale** IP (`100.x.y.z`), leave *Restrict to domain* **off**, and *Use with exit node* **off**. Save.
2. Turn on **“Override local DNS.”**

This is the [Chapter 4](#) Step 5 step. Verify it took with `tailscale dns status` on any device, under *Resolvers* you should see your Pi's `100.x.y.z`, not “no resolvers configured.” If it still says no resolvers, the admin-console setting hasn't applied yet.

The service map

Service	URL (everywhere on your tailnet)	What it does
Dashboard (Homepage)	<code>https://home.home</code> (or <code>https://homelab</code>)	Landing page; live tiles + links
Audiobookshelf	<code>https://abs.home</code>	Stream your audiobooks / Assimil

Service	URL (everywhere on your tailnet)	What it does
Pi-hole	<code>https://pihole.home</code>	DNS ad-blocking admin
Portainer	<code>https://homelab:9443</code>	Docker management GUI

All four work from any device signed into your tailnet, home Wi-Fi or cellular, with no IPs and no port numbers (except Portainer, which keeps its own :9443).

Verify it, layer by layer

Run these in order. Each isolates one layer, so the first failure pinpoints the problem.

1. The containers are all up (on the Pi):

```
docker ps --format "table {{.Names}}\t{{.Status}}"
```

You should see five **Up** containers: `caddy`, `homepage`, `pihole`, `portainer`, `audiobookshelf`.

2. They share the homelab network (on the Pi):

```
docker network inspect homelab --format "{{range .Containers}}{{.Name}}\n{{end}}"
```

All five names should appear. If `pihole` or `audiobookshelf` is missing, the reverse proxy can't reach it, re-read the network callout in [Chapter 4](#).

3. DNS resolves the pretty names (on the Pi):

```
dig +short pihole.home @127.0.0.1      # expect your Pi's Tailscale IP (100.x.y.z)
dig +short doubleclick.net @127.0.0.1 # expect 0.0.0.0 (ad-blocking works)
```

4. Caddy routes each name to the right service (on the Pi):

```
for name in pihole.home abs.home home.home; do
    printf "%-12s -> " "$name"
    curl -s -o /dev/null -w "HTTP %{http_code}\n" \
        --resolve "$name:80:${tailscale ip -4 | head -1}" "http://$name/"
done
```

Expect `pihole.home` -> HTTP 302 (it redirects to `/admin`) and `abs.home` / `home.home` -> HTTP 200. This tests the full chain, DNS name → Caddy on the Tailscale IP → the correct backend container, exactly as a remote device sees it.

5. The names work from another device (laptop/phone on the tailnet):

```
ping home.home          # answers from the Pi's 100.x.y.z
```

Then open `https://home.home`, `https://abs.home`, and `https://pihole.home` in a browser. If `ping` fails here but step 4 passed on the Pi, the missing piece is the **Tailscale DNS override** (the cloud setting at the top of this appendix).

6. The dashboard widgets have live data:

Open `https://home.home`. The Pi-hole tile should show today's blocked-query count and the Audiobookshelf tile your library, proof the widgets authenticated through the **homelab** network using the secrets in `~/dashboard/.env`. If a tile says "API error," see the widget troubleshooting in [Chapter 5](#) (for Pi-hole, the usual culprit is a missing `version: 6`).

What works away from home (and what deliberately doesn't)

Once the cloud setting above is on, here's the honest picture the moment you leave the house, the full reasoning is in [Chapter 7](#):

Mode (away)	Homelab URLs?	Pi-hole ad-block?	Hides your IP?
Tailscale on, no exit node (everyday default)	Yes	Yes	No
Tailscale + your Pi as exit node	Yes	Yes	No (home IP)
Tailscale + Mullvad exit node	Yes	No (Mullvad DNS)	Yes
Aura (or any standalone VPN) on	No	No	Yes

The one rule behind the table: a phone runs **one** VPN at a time, and whatever VPN is active owns DNS. So homelab access, Pi-hole filtering, and a privacy VPN can't all be on at once. You switch between coherent modes. The everyday default (top row) is the one you live in.

Reboot behavior

Nothing extra is required after a power cycle. Every container uses a `restart: policy` (`unless-stopped` or `always`), so Docker brings them all back on boot; the **homelab** network is persistent; and the `systemd-resolved` stub stays disabled (the drop-in from [Chapter 3](#) survives reboots), so Pi-hole reclaims port 53 cleanly. Reboot the Pi and re-run the verification steps above, every row should pass without you touching anything.